

Group Assignment

*Machine Learning on Social Media Posts:
Topic and Sentiment Analysis of Reddit Posts*

Program Name: MSc Artificial Intelligence

Module Name: Machine Learning and Python, AI 104

Author: Charalampos Lazaridis Patsalias

Group project (4 students). My role: group lead; I assigned the tasks and composed the final report. Grade: 85/100.

Contents

1	Data Collection	1
2	Data Annotation	2
3	Text Cleaning	3
4	Clustering by Topic	3
4.1	a. Clustering posts by topic and identifying important words	3
4.2	b and c. Assigning scores to important words based on frequency and creating the feature dataframe	4
5	Classification by Topic	5
6	Classification by Sentiment	12
6.1	Classification models	12
6.2	Training and testing	12
6.3	Model evaluation	12
6.4	Results and interpretation	13
6.4.1	a. Topic case	13
6.4.2	b. No topic case	13
7	Further Analysis	14
7.1	a. Temporal sentiment analysis	14
7.2	b. Association between topics and sentiment	17
7.3	c. Classification with Bigrams	18
7.4	d. Most frequent words and bigrams by positive and negative sentiment	21

List of Figures

1	Table with the post data as retrieved from Reddit.	1
2	Number of posts per topic.	2
3	Table of posts after the sentiment labels were attached by our fellow students. . .	2
4	The final table with the addition of the mean sentiment.	2
5	Bar chart of the most important words per topic.	4
6	Part of the input feature table based on the vocabulary for the case where the score depends on the topic. In the corresponding vocabulary that did not depend on the topic we had over 2500 words. This difference exists because we set min_df and max_df, and these variables differ depending on whether we consider all posts together or the posts of each topic separately.	5
7	Part of the code loading the datasets for the two cases.	5
8	Display of information about each dataset.	6
9	Model creation and definition of param_grid for selecting the best hyperparame- ters, topic case.	7

10	Model creation and definition of param_grid for selecting the best hyperparameters, no topic case.	7
11	Model evaluation results during training, no topic case.	8
12	Model evaluation results during training, topic case.	8
13	Code producing prediction results, topic case.	9
14	Code producing prediction results, no topic case.	10
15	Presentation of prediction results for both cases.	10
16	Heatmap of the confusion matrix for predictions on the test set, no topic case. .	11
17	Heatmap of the confusion matrix for predictions on the test set, topic case. . .	11
18	Loading and processing the data for the question.	14
19	Computation of weekly posts per topic and sentiment.	14
20	Creation of charts and weeks with the most positive/negative posts.	15
21	Weekly number of posts for AGI.	16
22	Weekly number of posts for climate action.	16
23	Weekly number of posts for nutrition in fitness.	16
24	Weekly number of posts for psychology.	16
25	Top 3 weeks for positive/negative posts per topic.	17
26	Heatmap representation of the contingency table.	18
27	Heatmap representation of the confusion matrix for topic classification, no topic case.	20
28	Heatmap representation of the confusion matrix for sentiment classification, no topic case.	20
29	Bar chart with the most frequent words and bigrams for negative sentiment. . .	22
30	Bar chart with the most frequent words and bigrams for positive sentiment. . .	22

List of Tables

1	Model evaluation table, topic case.	13
2	Model evaluation table, no topic case.	13

Introduction

This assignment aims to apply machine learning techniques to the analysis of posts from social media, specifically tweets and posts from Reddit [1], related to current topics. The assignment starts from data collection and proceeds to text cleaning, feature extraction, clustering, topic and sentiment classification, as well as further analyses.

The main goals of the study are:

- To collect a representative dataset for multiple topics.
- To annotate and clean the data so that it is suitable for analysis.
- To identify the most important words of each topic through clustering techniques and frequency analysis.
- To build and evaluate classification models both for topic categorisation and for sentiment categorisation (positive, negative, neutral).
- To carry out further analyses, such as the evolution of sentiment over time, the association of topics with sentiment, and the use of more complex features such as bi-grams.

In this way, the assignment incorporates core concepts of machine learning and NLP, such as text cleaning, feature extraction, building classification models, and evaluating those models. At the same time, extended analyses are carried out that allow a deeper understanding of the data and the patterns that appear.

1 Data Collection

For data collection we used the Reddit API through the praw library in Python [2]. We defined the subreddits corresponding to each selected topic and downloaded up to 1000 recent posts from each one. The topics and their posts are shown below. For each post, basic attributes were stored such as the title, the main text, the creation date and the number of comments.

	title	selftext	created_utc	text_length	topic
0	What's Cooking Wednesday - April 30, 2025	Post recipes, nutritional plans, favorite food...	2025-04-30 10:01:27	82	nutrition in fitness
1	Daily Simple Questions Thread	Welcome to our Daily Simple Questions thread -...	2025-05-09 12:00:39	434	nutrition in fitness
2	What's an underrated superfood?	Something like chia seeds, mushrooms, or sprou...	2025-06-25 05:18:24	84	nutrition in fitness
3	What to do with old fabrics?	I'll never forget the images of the parts of t...	2025-07-21 17:53:36	382	climate action
4	What are your go to breakfast recipes?	I am trying to think of some new easy breakfas...	2025-02-26 00:26:30	290	nutrition in fitness

Figure 1: Table with the post data as retrieved from Reddit.

Because the total number of posts was very large, we initially selected only 4 of the topics: Climate Action, Artificial General Intelligence, Nutrition in Fitness and Psychology. We kept only a percentage of these before handing them over for labelling. The number of posts per topic in the final dataframe is shown below.

topic	
climate action	671
nutrition in fitness	547
Artificial General Intelligence	492
Psychology	444

Figure 2: Number of posts per topic.

2 Data Annotation

We gave this dataframe to two teams to label according to the sentiment of each post. The dataframe after labelling is shown below.

	title	selftext	created_utc	topic	text_length	sentiment	sentiment_2	sentiment_3
0	Finding Researchers that incorporate Queer Stu...	Good evening everyone, I am wondering if anyon...	2020-11-07 06:03:22	Psychology	385	Positive	Positive	Neutral
1	Depression	Hi. I am battling with depression for four yea...	2025-07-24 08:00:35	Psychology	222	Negative	Negative	Negative
2	People are coming to stay Aug 8 and I'm too de...	Usually I can conceal my mental state because ...	2025-07-23 02:30:55	Psychology	478	Negative	Negative	Negative
3	Four days of cycling in the hilly west of Germ...	Coming from a relatively flat part of Belgium,...	2025-06-28 21:40:24	nutrition in fitness	270	Positive	Positive	Positive
4	Synoptic symbols font pack	Is anyone aware of a font pack that is availab...	2025-06-29 03:44:35	climate action	113	Neutral	Neutral	Neutral

Figure 3: Table of posts after the sentiment labels were attached by our fellow students.

From the 3 sentiment columns we created the final one (final sentiment) by applying the following method: if a sentiment appeared twice for a post, we placed that sentiment in the final sentiment; if all sentiments appeared once each, we placed Neutral in the final sentiment. The final dataframe on which we worked in the following questions is shown below.

	title	selftext	clean_text	created_utc	topic	text_length	sentiment	sentiment_2	sentiment_3	final_sentiment
0	Finding Researchers that incorporate Queer Stu...	Good evening everyone, I am wondering if anyon...	good evening wonder know program researcher in...	2020-11-07 06:03:22	Psychology	385	Positive	Positive	Neutral	Positive
1	Depression	Hi. I am battling with depression for four yea...	battle depression year naman hindi tanong lang...	2025-07-24 08:00:35	Psychology	222	Negative	Negative	Negative	Negative
2	People are coming to stay Aug 8 and I'm too de...	Usually I can conceal my mental state because ...	usually conceal mental state day adult kid pre...	2025-07-23 02:30:55	Psychology	478	Negative	Negative	Negative	Negative
3	Four days of cycling in the hilly west of Germ...	Coming from a relatively flat part of Belgium,...	come relatively flat belgium initially kind sc...	2025-06-28 21:40:24	nutrition in fitness	270	Positive	Positive	Positive	Positive
4	Synoptic symbols font pack	Is anyone aware of a font pack that is availab...	aware font pack available download contain sym...	2025-06-29 03:44:35	climate action	113	Neutral	Neutral	Neutral	Neutral

Figure 4: The final table with the addition of the mean sentiment.

3 Text Cleaning

To clean the texts we implemented a `clean_text` function in Python, based on spaCy [3], so that a normalised `clean_text` field is produced inside the dataframe that feeds the next steps. Specifically [4]:

- We remove URLs with regular expressions.
- We delete hashtags/mentions (tokens starting with # or @).
- We tokenise with spaCy’s `en_core_web_lg` model (disabling NER/Parser for speed).
- We remove stopwords, punctuation and whitespace.
- We apply lemmatisation and lowercasing.
- We filter tokens with additional quality rules: length ≤ 2 , presence of digits, non-alphanumeric characters, as well as words we placed on a blacklist such as “vs”, “etc”, “e.g”, “i.e”. Finally, we join the “clean” lemmas into a string that is used as input for feature extraction.

The cleaning aims at removing noise (e.g. URLs/mentions) and unifying forms (lemmatisation), so that sparsity is reduced and the semantic consistency of the features is improved. This facilitates both the frequency metrics and the TF-IDF representations that follow in the next stages.

4 Clustering by Topic

4.1 a. Clustering posts by topic and identifying important words

In the first sub-question we are asked to cluster the posts based on their topic and to identify the most important words per category.

To achieve this, we exploited the `clean_text` field produced after cleaning, and applied TF-IDF vectorisation [5]. This method transforms each text into a vector space, where each dimension corresponds to a word and its value represents its importance in the specific text. The term Term Frequency (TF) counts how many times a word appears in a text, while Inverse Document Frequency (IDF) reduces the weight of words that appear very frequently across all texts (e.g. “people”, “time”) and therefore do not offer substantial discrimination.

After the TF-IDF vectorizer was applied, we were able to identify the words with the highest scores per topic, that is, those that best differentiate the topic from the rest. These words and their importance based on their TF-IDF score can be seen in the following horizontal bar chart.

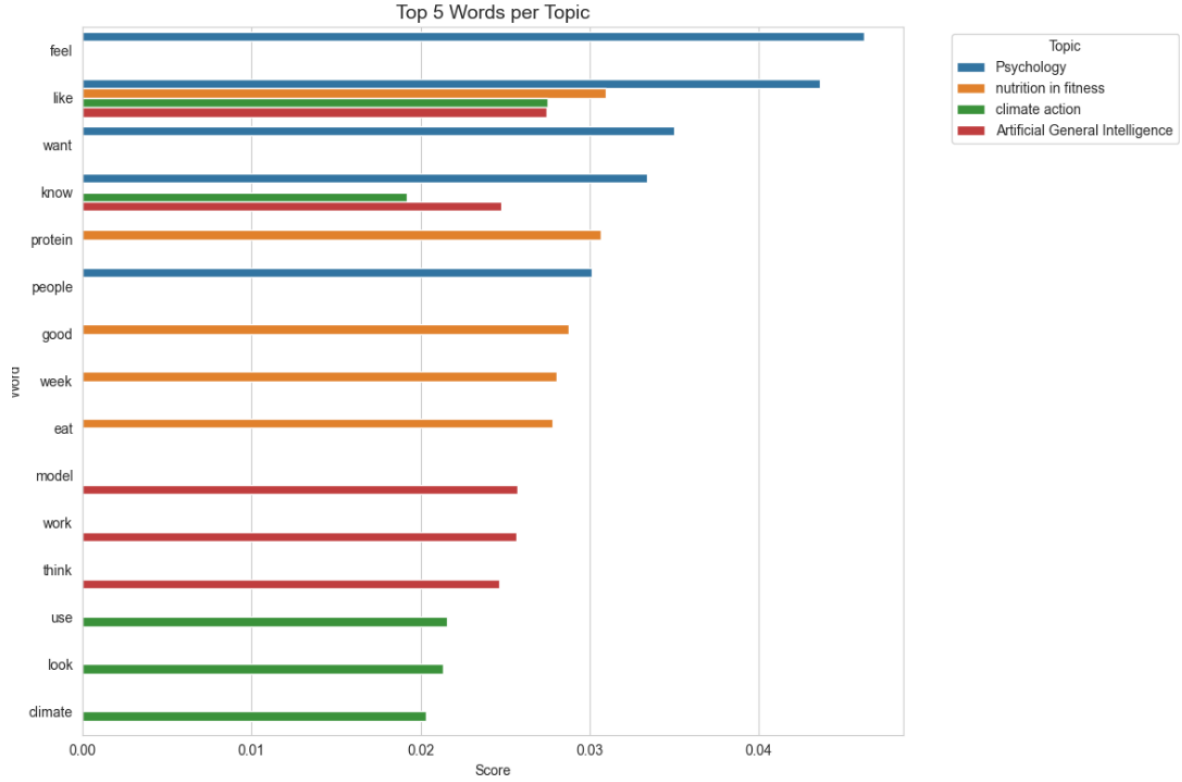


Figure 5: Bar chart of the most important words per topic.

As can be seen, there are words such as *like* that are important for all topics, but also words such as *feel* that are very important only in specific topics. These are the ones that could help greatly in a topic classification task.

4.2 b and c. Assigning scores to important words based on frequency and creating the feature dataframe

For the second sub-question we did not use a predefined “scoring system” (e.g. 1-3-5), but worked directly with the real word-frequency data. Specifically, we implemented a function that computes, for each post, how many times each vocabulary word appears in that post, and to obtain the score we divided this number by the total number of words in the post. The result was a table where the columns correspond to words and the rows to texts, while the values express the score of the word in each post. In this way, instead of translating the frequencies into “categories”, we kept the numeric information and processed it so that it is more useful for the next classification steps. Part of the final result is shown below:

	dedicated	hot	sit	love	track	power	craving	respond	appreciate	benefit	...	historical	feedback	news	tech
0	0.0	0.0	0.0	0.0	0.0	0.0000	0.0	0.0	0.0357	0.0	...	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	0.0000	0.0	0.0	0.0000	0.0	...	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0000	0.0	0.0	0.0000	0.0	...	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0455	0.0	0.0	0.0000	0.0	...	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0000	0.0	0.0	0.0000	0.0	...	0.0	0.0	0.0	0.0

5 rows × 1982 columns

Figure 6: Part of the input feature table based on the vocabulary for the case where the score depends on the topic. In the corresponding vocabulary that did not depend on the topic we had over 2500 words. This difference exists because we set `min_df` and `max_df`, and these variables differ depending on whether we consider all posts together or the posts of each topic separately.

As we can see, most words have the value zero, which is expected since each word appears in only a few texts.

5 Classification by Topic

To predict the topic classification of the posts we applied a plethora of different algorithms with two strategies for creating the features. These two strategies are creating features ignoring the topics (no topic case) and creating them taking the topics into account (topic case), Figure 7. This aims at evaluating the performance of the algorithms on the common problem of performance degradation due to many sparse dimensions, since ignoring the topics we have a set of 3127 features, versus 2283, Figure 8 [6].

```

Python
FeaturesPerTopic = pd.read_csv("tf_idf_features_per_topic.tsv", sep="\t").drop("sentiment_label", axis=1)
FeaturesAll = pd.read_csv("tf_idf_features_per_document.tsv", sep="\t").drop("sentiment_label", axis=1)
Python

```

Figure 7: Part of the code loading the datasets for the two cases.

```
FeaturesAll.info()
FeaturesPerTopic.info()
✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2154 entries, 0 to 2153
Columns: 3129 entries, abilities to topic_label
dtypes: float64(3127), object(2)
memory usage: 51.4+ MB

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2154 entries, 0 to 2153
Columns: 2285 entries, ability to yogurt
dtypes: float64(2283), object(2)
memory usage: 37.6+ MB
```

Figure 8: Display of information about each dataset.

The classification algorithms we apply are Naive Bayes, KNN, Decision Tree, Logistic Regression, SVM and Random Forest. All of them are referenced in the literature for their effectiveness in text classification problems, with positive and negative aspects as well as specific assumptions. For example, Naive Bayes performs well despite the violation of the feature independence assumption, SVM with the linearity assumption has better results, and KNN succeeds in classification but with a higher time cost [6]. For selecting the hyperparameters of each algorithm we use the Grid Search Cross Validation technique, Figures 9 and 10, which through an iterative procedure of evaluating combinations of hyperparameter values on the training data gives the optimal choices [7]. The best result for each algorithm is presented in Figures 11 and 12, with the important observation that all proposed models differ between the two strategies, with the exception of Decision Tree and Random Forest.

```

seed = 42
X_perTopic = FeaturesPerTopic.loc[:, FeaturesPerTopic.columns.difference(['source_text', 'topic_label'])]
y_perTopic = FeaturesPerTopic['topic_label']

X_train_perTopic, X_test_perTopic, y_train_perTopic, y_test_perTopic = train_test_split(
    X_perTopic, y_perTopic, test_size=0.2, random_state=seed, stratify=y_perTopic
)

pipe_perTopic = Pipeline([("clf", MultinomialNB())])
param_grid_perTopic = [
    {"clf": [MultinomialNB()], "clf__alpha": [0.1, 0.3, 0.5, 0.7, 1.0, 2.0]},
    {"clf": [KNeighborsClassifier()], "clf__n_neighbors": [20, 30, 40, 45, 50]},
    {"clf": [DecisionTreeClassifier(random_state=seed)],
     "clf__max_depth": [None, 10, 30],
     "clf__min_samples_split": [2, 5, 10]},
    {"clf": [SVC(random_state=seed)],
     "clf__C": [0.1, 0.5, 1, 4, 10],
     "clf__gamma": [0.1, 0.5, 1, 4, 10],
     "clf__kernel": ["linear", "poly", "rbf"]},
    {"clf": [LogisticRegression(max_iter=2000, solver="saga", n_jobs=-1, random_state=seed)],
     "clf__C": [0.1, 1, 3]},
    {"clf": [RandomForestClassifier(random_state=seed)],
     "clf__n_estimators": [100, 500],
     "clf__max_depth": [5, 15, None]}
]

cv_perTopic = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed)
gs_perTopic = GridSearchCV(pipe_perTopic, param_grid_perTopic, scoring="f1_weighted", cv=cv_perTopic, refit=True)
gs_perTopic.fit(X_train_perTopic, y_train_perTopic)

```

Figure 9: Model creation and definition of param_grid for selecting the best hyperparameters, topic case.

```

seed = 42
X_All = FeaturesAll.loc[:, FeaturesAll.columns.difference(['source_text', 'topic_label'])]
y_All = FeaturesAll['topic_label']

X_train_All, X_test_All, y_train_All, y_test_All = train_test_split(
    X_All, y_All, test_size=0.2, random_state=seed, stratify=y_All
)

pipe_All = Pipeline([("clf", MultinomialNB())])

param_grid_All = [
    {"clf": [MultinomialNB()], "clf__alpha": [0.1, 0.3, 0.5, 0.7, 1.0, 2.0]},
    {"clf": [KNeighborsClassifier()], "clf__n_neighbors": [20, 30, 40, 45, 50]},
    {"clf": [DecisionTreeClassifier(random_state=seed)],
     "clf__max_depth": [None, 10, 30],
     "clf__min_samples_split": [2, 5, 10]},
    {"clf": [SVC(random_state=seed)],
     "clf__C": [0.1, 0.5, 1, 4, 10],
     "clf__gamma": [0.1, 0.5, 1, 4, 10],
     "clf__kernel": ["linear", "poly", "rbf"]},
    {"clf": [LogisticRegression(max_iter=2000, solver="saga", n_jobs=-1, random_state=seed)],
     "clf__C": [0.1, 1, 3]},
    {"clf": [RandomForestClassifier(random_state=seed)],
     "clf__n_estimators": [100, 500],
     "clf__max_depth": [5, 15, None]}
]

cv_All = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed)
gs_All = GridSearchCV(pipe_All, param_grid_All, scoring="f1_weighted", cv=cv_All, refit=True)
gs_All.fit(X_train_All, y_train_All)

```

Figure 10: Model creation and definition of param_grid for selecting the best hyperparameters, no topic case.

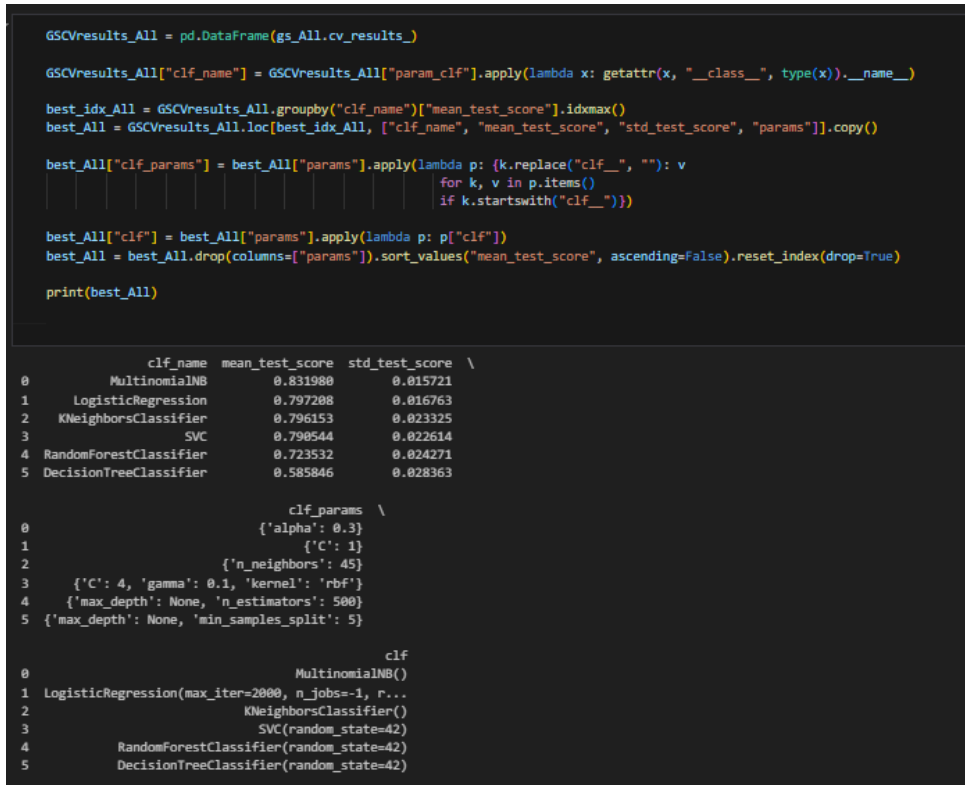


Figure 11: Model evaluation results during training, no topic case.

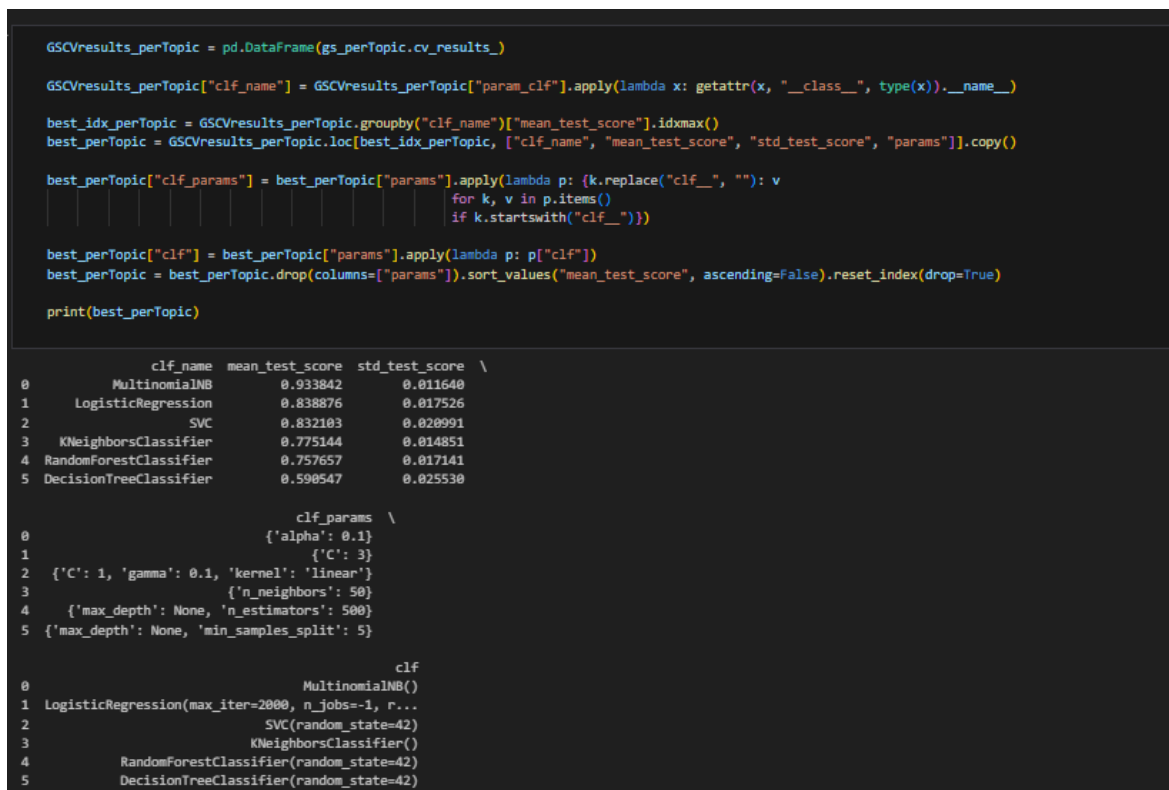


Figure 12: Model evaluation results during training, topic case.

Finally, applying the models to the test data, Figures 13 and 14, and computing the metrics accuracy, precision, recall and F1 score, Figure 15, we draw the following conclusions:

- The best model regardless of strategy is Naive Bayes. The large and sparse dimensions of the features are also the reason that allows it to deliver these performances.
- Logistic Regression and SVM follow relatively close in performance behind Naive Bayes, since linear relationships in text data are a characteristic they can exploit. We also observe the negative effect of choosing a non-linear kernel (rbf) for SVM in the strategy that ignores the topics, which ranks it below KNN.
- For KNN, we see that it performs better in the strategy with the overall features, given that a high k with a larger number of features enables better prediction.
- Random Forest and Decision Tree have the lowest results, since compared to the other models they do not have such strong assumptions and their simplicity is their main disadvantage.
- The results of the feature-creation strategy that includes the topics are higher. However, this only proves that the performances are inflated, because most features originate from one topic only, giving the models an advantage in recognising each topic.

The results for the metrics are fully confirmed by the corresponding confusion matrices for Naive Bayes, Figures 16 and 17, with a small confusion in the predictions between AGI and Psychology.

```

results_perTopic = []

for _, row in best_perTopic.iterrows():
    base = row["clf"]
    params = row["clf_params"]

    model = clone(base).set_params(**params)
    model.fit(X_train_perTopic, y_train_perTopic)
    y_pred_perTopic = model.predict(X_test_perTopic)

    acc = accuracy_score(y_test_perTopic, y_pred_perTopic)
    prec, rec, f1, _ = precision_recall_fscore_support(
        y_test_perTopic, y_pred_perTopic, average="weighted", zero_division=0
    )

    results_perTopic.append({
        "Model": row["clf_name"],
        "Params": params,
        "Accuracy": acc,
        "Precision_weighted": prec,
        "Recall_weighted": rec,
        "F1_weighted": f1
    })

results_df_perTopic = pd.DataFrame(results_perTopic).sort_values("F1_weighted", ascending=False)
print(results_df_perTopic)

```

Figure 13: Code producing prediction results, topic case.

```

results_All = []

for _, row in best_All.iterrows():
    base = row["clf"]
    params = row["clf_params"]

    model = clone(base).set_params(**params)
    model.fit(X_train_All, y_train_All)
    y_pred_All = model.predict(X_test_All)

    acc = accuracy_score(y_test_All, y_pred_All)
    prec, rec, f1, _ = precision_recall_fscore_support(
        y_test_All, y_pred_All, average="weighted", zero_division=0
    )

    results_All.append({
        "Model": row["clf_name"],
        "Params": params,
        "Accuracy": acc,
        "Precision_weighted": prec,
        "Recall_weighted": rec,
        "F1_weighted": f1
    })

results_df_All = pd.DataFrame(results_All).sort_values("F1_weighted", ascending=False)
print(results_df_All)

```

Figure 14: Code producing prediction results, no topic case.

```

cols = ("Model": "Model",
        "Params": "Parameters",
        "Accuracy": "Accuracy",
        "Precision_weighted": "Precision",
        "Recall_weighted": "Recall",
        "F1_weighted": "F1 Score")

metrics = ["Accuracy", "Precision", "Recall", "F1 Score"]

# Tekird results_df_All
# Petovagasta onluw
results_df_All = results_df_All.rename(columns = cols)
# Itpoyvukomaton metrics
results_df_All[metrics] = results_df_All[metrics].astype(float).round(3)
# Iotvayum pu 80m to F1 Score
results_df_All = results_df_All.sort_values("F1 Score", ascending=False).reset_index(drop=True)

display(results_df_All.style.hide(axis="index").set_table_attributes("class='table'").set_caption("Classification results - Vectorization irrespective of Topics").format("{:.3f}", subset=metrics))

# Tekird results_df_perTopic
# Petovagasta onluw
results_df_perTopic = results_df_perTopic.rename(columns = cols)
# Itpoyvukomaton metrics
results_df_perTopic[metrics] = results_df_perTopic[metrics].astype(float).round(3)
# Iotvayum pu 80m to F1 Score
results_df_perTopic = results_df_perTopic.sort_values("F1 Score", ascending=False).reset_index(drop=True)

display(results_df_perTopic.style.hide(axis="index").set_table_attributes("class='table'").set_caption("Classification results - Vectorization per Topic").format("{:.3f}", subset=metrics))

```

Classification results - Vectorization irrespective of Topics					
Model	Parameters	Accuracy	Precision	Recall	F1 Score
MultinomialNB	{alpha: 0.3}	0.861	0.861	0.861	0.861
LogisticRegression	{C: 1}	0.814	0.816	0.814	0.813
KNeighborsClassifier	{n_neighbors: 45}	0.805	0.813	0.805	0.804
SVC	{C: 4, gamma: 0.1, kernel: 'rbf'}	0.800	0.803	0.800	0.801
RandomForestClassifier	{max_depth: None, n_estimators: 500}	0.759	0.762	0.759	0.756
DecisionTreeClassifier	{max_depth: None, min_samples_split: 5}	0.654	0.656	0.654	0.652

Classification results - Vectorization per Topic					
Model	Parameters	Accuracy	Precision	Recall	F1 Score
MultinomialNB	{alpha: 0.1}	0.954	0.954	0.954	0.954
LogisticRegression	{C: 3}	0.861	0.860	0.861	0.860
SVC	{C: 1, gamma: 0.1, kernel: 'linear'}	0.842	0.844	0.842	0.843
RandomForestClassifier	{max_depth: None, n_estimators: 500}	0.803	0.806	0.803	0.803
KNeighborsClassifier	{n_neighbors: 50}	0.780	0.785	0.780	0.781
DecisionTreeClassifier	{max_depth: None, min_samples_split: 5}	0.682	0.690	0.682	0.684

Figure 15: Presentation of prediction results for both cases.

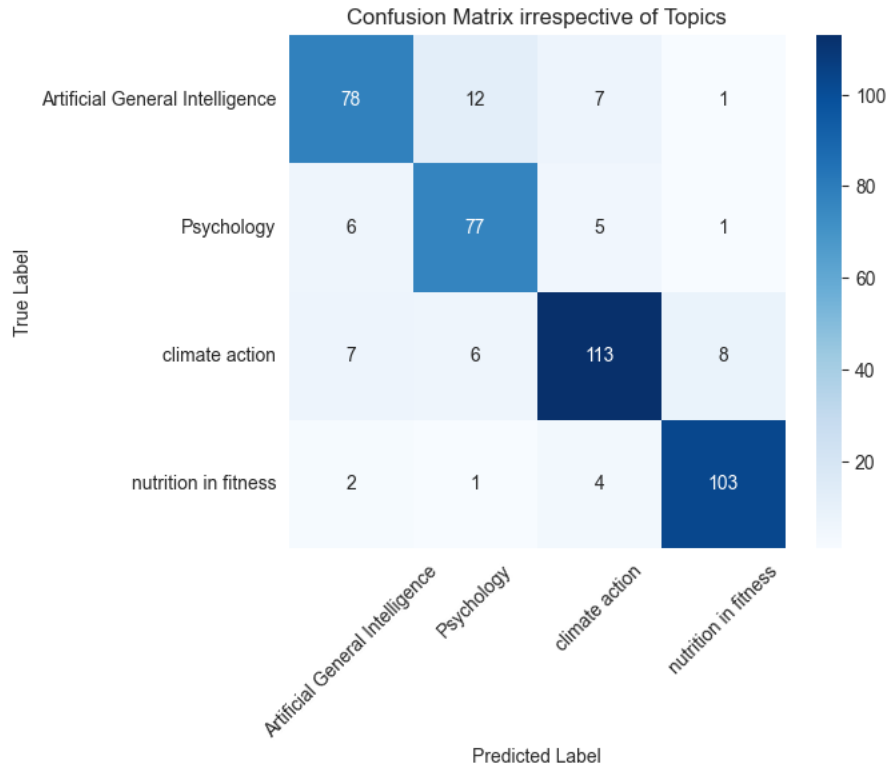


Figure 16: Heatmap of the confusion matrix for predictions on the test set, no topic case.

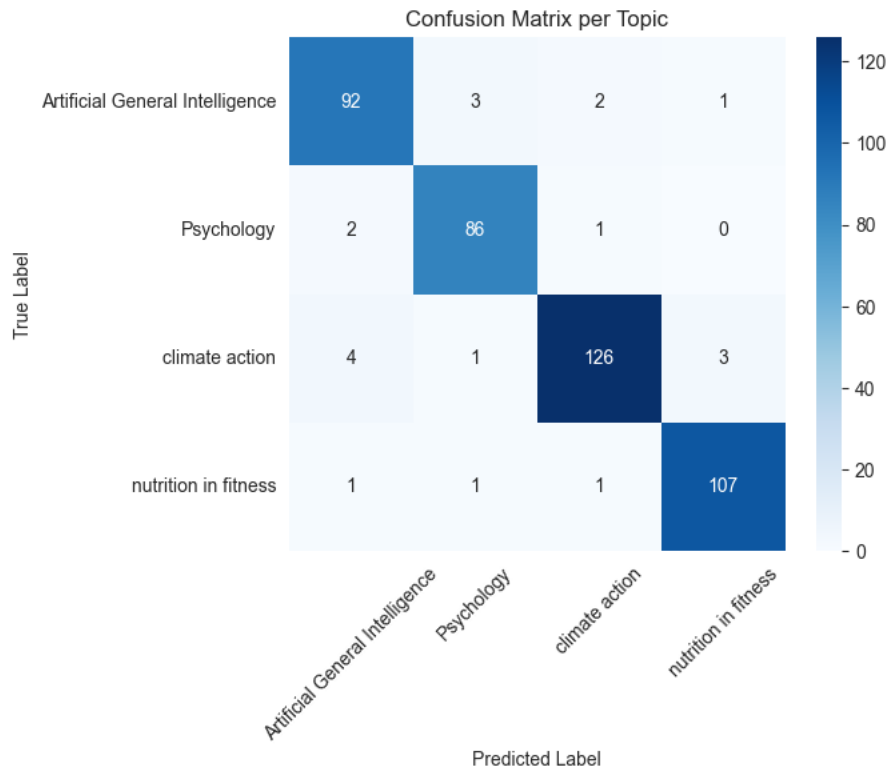


Figure 17: Heatmap of the confusion matrix for predictions on the test set, topic case.

6 Classification by Sentiment

This section focuses on sentiment classification. To categorise the posts as positive or negative, two feature sets were created, per document and per topic, using TF-IDF vectorisation. A data preprocessing step was carried out so that posts with neutral sentiment were removed, and each post was represented as a numeric feature vector using TF-IDF. Then various supervised machine learning models were developed with Scikit-learn [8], with the sentiment label acting as the target variable.

6.1 Classification models

Four different models were used and compared [9]:

- Naive Bayes, which is simple, fast and based on the independence assumption between features.
- K-Nearest Neighbors (KNN), where classification depends on the majority class among the nearest samples in the feature space.
- Support Vector Machines (SVM), which try to find the optimal hyperplane that separates the sentiment classes.
- Random Forests, an ensemble of decision trees that improves generalisation and robustness.

Each model was embedded in a pipeline that included hyperparameter optimisation. In this way it was ensured that the models were comparable and tuned for the best possible performance.

6.2 Training and testing

The dataset was split into a training set and a test set with an 80/20 ratio. The training set was used to train the models, while the test set was kept for evaluation. This split ensures that the performance metrics reflect how well the models generalise to data they have not “seen” before.

6.3 Model evaluation

An evaluation procedure was applied for each model, measuring the following [10]:

- Accuracy: Measures the percentage of posts classified correctly, reflecting the overall performance of the model.
- Precision: Shows the percentage of posts predicted as belonging to a sentiment that actually belong to it, measuring the reliability of the model’s predictions.
- Recall: Measures the percentage of actual posts of a specific sentiment that are correctly identified, indicating the model’s ability to detect all relevant cases.
- F1-score: Combines precision and recall through their harmonic mean, providing a balanced metric of classification performance.
- Confusion Matrix: Presents the number of true positives, false positives, true negatives and false negatives, highlighting specific types of prediction errors.

- Training Time: Represents the time required to train the model, giving an indication of its computational efficiency.

6.4 Results and interpretation

6.4.1 a. Topic case

Random Forest achieved the highest accuracy (0.748) and performed well on positive posts, but struggled with negative ones, while requiring a longer training time (23.59 seconds). Naive Bayes was very fast (0.09 seconds) and had satisfactory performance, albeit with lower accuracy (0.724). SVM showed strong performance on positive posts (high recall), but had a slower training time (29.07 seconds). Finally, KNN was the least balanced, with low performance on negative posts and a moderate training time (0.65 seconds).

Model	Accuracy	Precision (Pos-Neg)	Recall (Pos-Neg)	F1-score (Pos-Neg)	Training Time
Random Forest	0.748	0.74 - 0.82	0.96 - 0.33	0.84 - 0.47	23.59
Naive Bayes	0.724	0.78 - 0.60	0.82 - 0.54	0.80 - 0.57	0.09
SVM	0.741	0.74 - 0.77	0.95 - 0.34	0.83 - 0.47	29.07
KNN	0.627	0.74 - 0.46	0.68 - 0.52	0.71 - 0.49	0.65

Table 1: Model evaluation table, topic case.

6.4.2 b. No topic case

In this case Random Forest showed the highest accuracy (0.779) and a good balance between the classes, achieving very high recall for positive posts (0.96), but moderate performance on negative ones (recall 0.43), while it had the longest training time (40.42 seconds). Naive Bayes remained extremely fast (0.11 seconds), with an overall accuracy of 0.711 and relatively low performance on the negative class (precision 0.59, recall 0.53). SVM achieved high accuracy (0.752), with balanced behaviour and better performance on negative posts compared to Naive Bayes (recall 0.57), although the training time was long (41.36 seconds). Finally, KNN reached an accuracy of 0.762, better than Naive Bayes and similar to SVM, with notable recall for positive posts (0.93), but limited ability to recognise negative ones (recall 0.45), while the training time remained relatively small (0.77 seconds).

Model	Accuracy	Precision (Pos-Neg)	Recall (Pos-Neg)	F1-score (Pos-Neg)	Training Time
Random Forest	0.779	0.76 - 0.86	0.96 - 0.43	0.85 - 0.57	40.42
Naive Bayes	0.711	0.77 - 0.59	0.81 - 0.53	0.79 - 0.56	0.11
SVM	0.752	0.79 - 0.66	0.85 - 0.57	0.82 - 0.61	41.36
KNN	0.762	0.76 - 0.77	0.93 - 0.45	0.84 - 0.56	0.77

Table 2: Model evaluation table, no topic case.

Comparing the two sets of results, it is observed that in the second case most models showed improved overall accuracy, with Random Forest increasing from 0.748 to 0.779, SVM from 0.742 to 0.752 and KNN from 0.628 to 0.762, while Naive Bayes showed a small drop (0.725 to 0.711).

At the same time, the ability to recognise negative posts improved noticeably for SVM and KNN, which showed higher recall compared to the first case, while Random Forest, despite achieving the highest overall accuracy, continued to have low recall for the negative class. In terms of training time, all models were slower in the second case, with Random Forest and SVM exceeding 40 seconds, which indicates greater computational complexity, while Naive Bayes and KNN remained quite time-efficient. Overall, the results show that there was generally an improvement in the models at the cost of time.

7 Further Analysis

7.1 a. Temporal sentiment analysis

For the analysis of sentiment over time per topic, we create a DataFrame keeping the necessary columns `clean_text`, `final_sentiment`, `topic`, `created_utc`, making the relevant conversions for the correct data type and ensuring that the content of the character columns is in the same format everywhere, Figure 20. In addition, we remove records with neutral sentiment, and compute the weekly sums per topic and positive/negative sentiment, Figure 21.

```
# Μεταφόρτωση του τελικού πίνακα με τα καθορισμένα κείμενα και τα τελικά συναισθήματα
path = "Final_assignment_dataframe_with_clean_text_and_final_sentiment.xlsx"
cols = ["clean_text", "final_sentiment", "topic", "created_utc"]
sent_analysis = pd.read_excel(path, usecols=cols).copy()
# Μετατροπή της στήλης created_utc σε datetime
sent_analysis["created_utc"] = pd.to_datetime(sent_analysis["created_utc"], errors="coerce", utc=True)
# Αφαίρεση γραμμών με NaN σε οποιαδήποτε από τις στήλες
sent_analysis = sent_analysis.dropna().reset_index(drop=True)
# Μετατροπή των τιμών των στηλών topic και final_sentiment σε πεζά και αφαίρεση κενών
sent_analysis[["topic", "final_sentiment"]] = (sent_analysis[["topic", "final_sentiment"]].astype("string").apply(lambda x: x.str.lower().str.strip()))
```

Figure 18: Loading and processing the data for the question.

```
# Αφαίρεση των γραμμών με neutral συναισθημα
sent_analysis = sent_analysis[sent_analysis["final_sentiment"] != "neutral"]

# Ομαδοποίηση ανά εβδομάδα, topic και συναισθημα
weekly = (
    sent_analysis
    .groupby(["topic", "final_sentiment",
              pd.Grouper(key="created_utc", freq="W-MON")])
    .size()
    .reset_index(name="count")
    .sort_values(["topic", "created_utc"])
)

# Pivot table για να έχουμε τις μετρήσεις ανά συναισθημα σε ξεχωριστές στήλες
weekly_counts = (
    weekly.pivot_table(index=["topic", "created_utc"],
                       columns="final_sentiment",
                       values="count")
    .reset_index()
    .sort_values(["topic", "created_utc"])
)
```

Figure 19: Computation of weekly posts per topic and sentiment.

Finally, we create the charts for positive and negative sentiment, and identify the 3 weeks with the most positive and negative posts per topic, Figure 22.

```

import matplotlib.pyplot as plt
import matplotlib.dates as mdates

topics = weekly_counts["topic"].astype(str).unique()
for t in topics:
    sub = (weekly_counts[weekly_counts["topic"] == t]
           .sort_values("created_utc")
           .copy())

    fig, ax = plt.subplots(figsize=(15, 4.5))
    ax.plot(sub["created_utc"].dt.tz_localize(None), sub["positive"], label="Positive")
    ax.plot(sub["created_utc"].dt.tz_localize(None), sub["negative"], label="Negative")
    # Hellenism: too slow + you can't get the epoch
    week_every = 6 # label every week
    anchor = mdates.MONDAY
    ax.xaxis.set_major_locator(mdates.WeekdayLocator(byweekday=anchor, interval=week_every))
    ax.xaxis.set_minor_locator(mdates.WeekdayLocator(byweekday=anchor, interval=week_every)) # light grid each week
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m-%d")) # show week start date
    plt.setp(ax.get_xticklabels(), rotation=45, ha="right")
    ax.grid(True, axis="x", which="both", alpha=0.15)
    ax.set_title(f"Weekly sentiment counts for {t}")
    ax.set_xlabel("Start of Week")
    ax.set_ylabel("Count")
    ax.legend(loc="best")
    fig.tight_layout()
    plt.show()

# Εύρησκυν των 3 ημερολόγιων με τις ημερολόγια θετικές και αρνητικές αναρτήσεις ανά topic
rows = []
for t in topics:
    sub = (weekly_counts[weekly_counts["topic"] == t]
           .sort_values("created_utc")
           .copy())
    # Εύρησκυν με τις 3 ημερολόγια θετικές αναρτήσεις
    pos_top = sub[sub["positive"] > 0].nlargest(3, "positive")[["created_utc", "positive"]]
    for _, r in pos_top.iterrows():
        rows.append({
            "topic": t,
            "sentiment": "positive",
            "week_start_utc": r["created_utc"].strftime("%Y-%m-%d"),
            "count": int(r["positive"])
        })
    # Εύρησκυν με τις 3 ημερολόγια αρνητικές αναρτήσεις
    neg_top = sub[sub["negative"] > 0].nlargest(3, "negative")[["created_utc", "negative"]]
    for _, r in neg_top.iterrows():
        rows.append({
            "topic": t,
            "sentiment": "negative",
            "week_start_utc": r["created_utc"].strftime("%Y-%m-%d"),
            "count": int(r["negative"])
        })
peaks_df = pd.DataFrame(rows).sort_values(
    ["topic", "sentiment", "count"], ascending=[True, True, False]
).reset_index(drop=True)
display(peaks_df)

```

Figure 20: Creation of charts and weeks with the most positive/negative posts.

The conclusions we can draw are:

- There is no clear seasonality for any topic and positive sentiments seem to prevail over negative ones, with the exception of Psychology for 2025, Figure 27. Also, the discontinuity of the plots is due to the design choice that weeks without posts are not replaced with zero points.
- AGI, Figure 23, sparse mentions until 2025, followed by a sharp increase. Positive posts dominate during the rise, with negative ones also increasing at a lower rate.
- Climate action, Figure 24, until the sharp rise in 2025 in both positive and negative posts, very few positive posts are observed. The multiple peaks for 2025 indicate ongoing discussion rather than a single week of extreme values.
- Nutrition in fitness, Figure 25, similar behaviour to climate action, with the difference that the intensity of the discussions in 2025 is lower.
- Psychology, Figure 26, a relatively low number of posts, mostly of positive sentiment, for most of the period. A single extreme negative-sentiment value is observed in August 2025, which may be worth further investigation as it may not reflect reality.

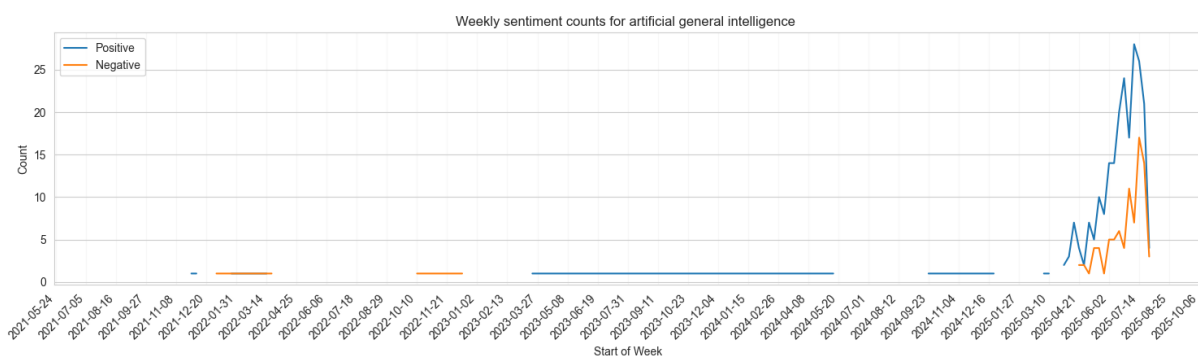
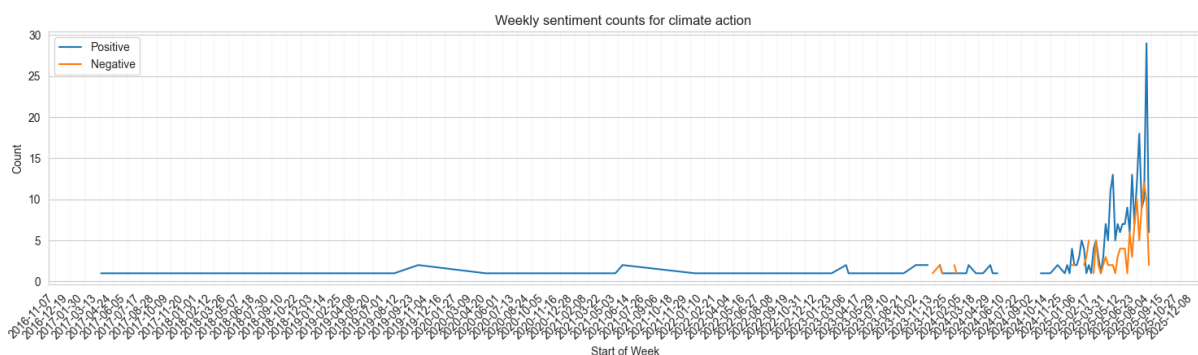


Figure 21: Weekly number of posts for AGI.



	topic	sentiment	week_start_utc	count
0	artificial general intelligence	negative	2025-07-14	17
1	artificial general intelligence	negative	2025-07-21	14
2	artificial general intelligence	negative	2025-06-30	11
3	artificial general intelligence	positive	2025-07-07	28
4	artificial general intelligence	positive	2025-07-14	26
5	artificial general intelligence	positive	2025-06-23	24
6	climate action	negative	2025-07-14	12
7	climate action	negative	2025-06-23	10
8	climate action	negative	2025-07-21	10
9	climate action	positive	2025-07-21	29
10	climate action	positive	2025-06-30	18
11	climate action	positive	2025-04-14	13
12	nutrition in fitness	negative	2025-05-26	7
13	nutrition in fitness	negative	2025-06-23	7
14	nutrition in fitness	negative	2025-06-30	4
15	nutrition in fitness	positive	2025-06-09	23
16	nutrition in fitness	positive	2025-06-30	22
17	nutrition in fitness	positive	2025-07-21	21
18	psychology	negative	2025-07-28	136
19	psychology	negative	2025-07-21	18
20	psychology	negative	2025-03-10	3
21	psychology	positive	2025-07-28	19
22	psychology	positive	2025-02-03	5
23	psychology	positive	2025-03-24	5

Figure 25: Top 3 weeks for positive/negative posts per topic.

7.2 b. Association between topics and sentiment

To examine whether there is an association between topics and sentiment, we created a contingency table recording how many positive, negative and neutral posts correspond to each topic. This table is shown in a heatmap representation below.

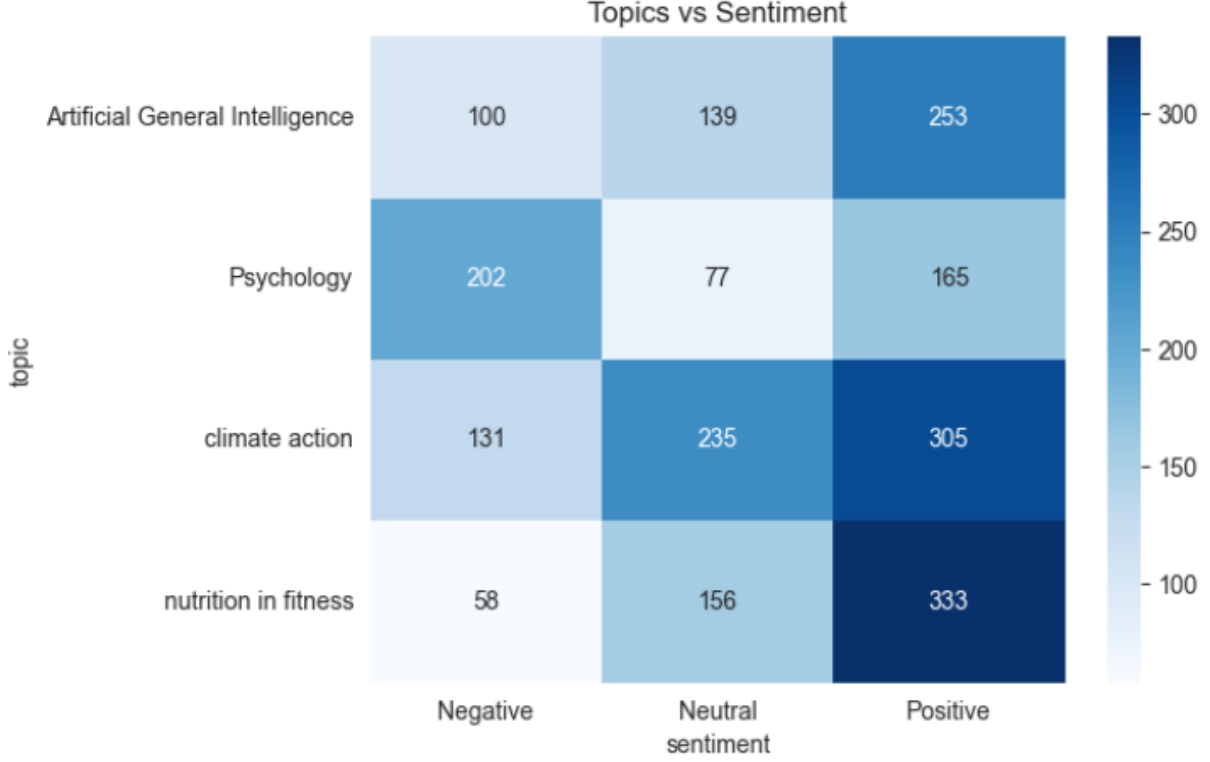


Figure 26: Heatmap representation of the contingency table.

From the heatmap we can see that specific topics such as psychology have a greater concentration of negative sentiments, while others, such as nutrition in fitness, tend to be associated more with positive posts.

We then applied a χ^2 test (chi-square test) [11] to determine whether the distribution of sentiments depends on the topic or is independent of it. The results showed a value of $\chi^2 \approx 200.93$ with p-value $\approx 1.2 \times 10^{-40}$, which means that the probability of the association being random is practically zero. So we can say with confidence that there is an association between them.

To estimate the strength of the association, we computed Cramér's V [11], which took a value of ≈ 0.216 . This value indicates a moderate association between sentiment and topic, that is, certain topics are more strongly associated with positive or negative sentiments.

7.3 c. Classification with Bigrams

In the penultimate part of the further analysis, we tried to replace the words that had previously been used as input features for training the models with bigrams [12].

For this purpose we applied the TF-IDF vectorizer for bigrams. In the case of topic classification, TF-IDF was applied separately for each topic and then the results were merged into a single dataframe. In contrast, for sentiment classification TF-IDF was applied to all posts together. This separation was made because in the previous questions we saw that:

- In topic classification, the best results were obtained when the features were created per topic and then composed into an integrated table.
- In sentiment classification, the highest performance was achieved when TF-IDF was com-

puted over all texts simultaneously.

This happens because in topic classification it is important to highlight the distinctive lexical patterns of each topic. If we apply TF-IDF to all texts together, some words or bigrams that are characteristic of only one topic may be “lost” due to the overall distribution across the whole set. Thus, per-topic TF-IDF ensures that each topic will be represented accurately by its own most discriminative bigrams.

In contrast, in sentiment classification the goal is to identify expressions that denote positive or negative sentiment, regardless of topic. Phrases such as “thank you so much” or “hard time” may appear in any topic. Therefore, it is preferable for TF-IDF to be computed over all documents together, so that the model has a unified vocabulary of sentiment expressions that is not limited to the context of one topic.

According to the results of the previous questions:

- For topic classification (Question 5), the best model with unigrams was Multinomial Naive Bayes, which showed the highest F1-score and an overall balanced performance.
- For sentiment classification (Question 6), the best model with unigrams was Random Forest, which excelled in accuracy and F1-score, although it had a higher training time compared to the others.

We now retrained the same models, this time with bigram features, so that a direct comparison could be made. The results showed:

- For topic classification, accuracy with bigrams dropped to 75%, significantly lower than the performance with words ($\approx 95\%$).
- For sentiment classification, accuracy with bigrams was about 69%, lower than the corresponding one with words ($\approx 78\%$).

The results of the two classifications are shown in more detail in the following confusion matrices.

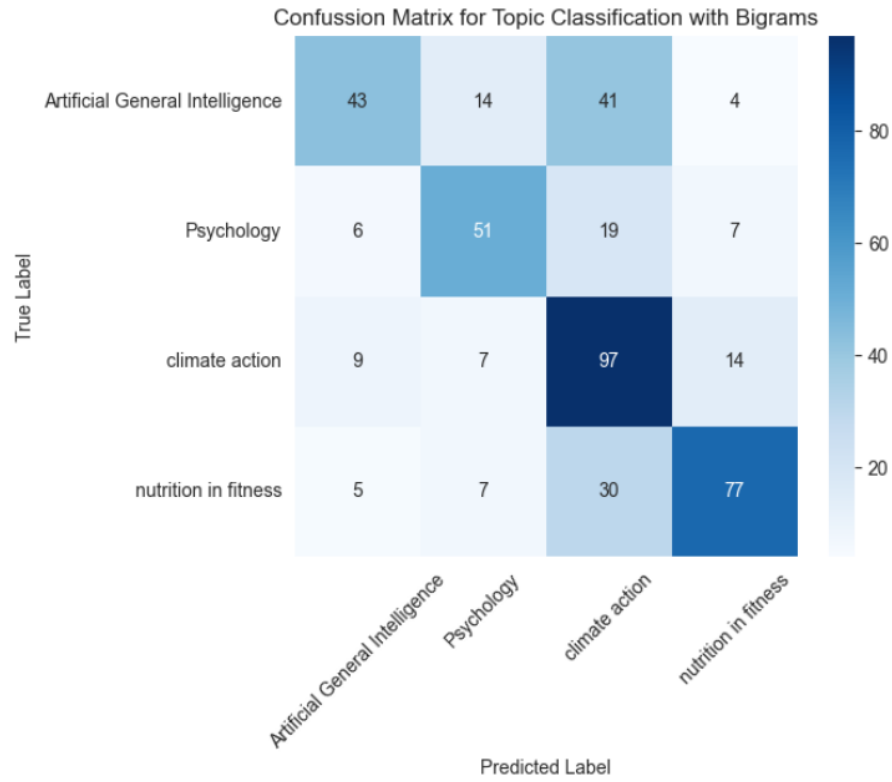


Figure 27: Heatmap representation of the confusion matrix for topic classification, no topic case.

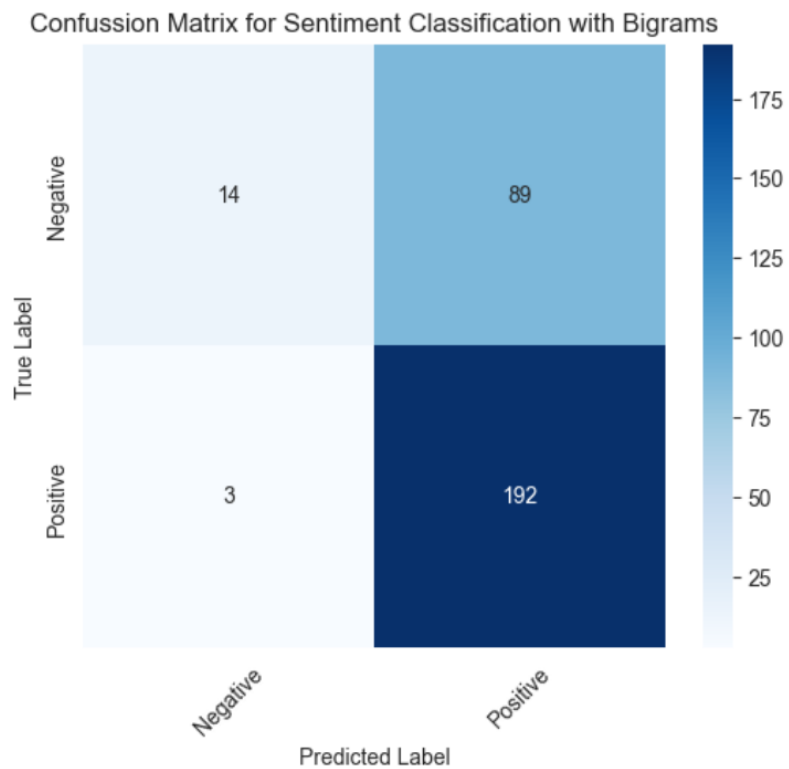


Figure 28: Heatmap representation of the confusion matrix for sentiment classification, no topic case.

The lower performance of bigrams in our case is mainly due to the combination of the size

and particularities of the dataset. The final dataset included 2,154 texts, which are relatively few to support a vocabulary with thousands of bigrams. Although we limited the features to 4,000 and set an occurrence threshold of `min_df=2`, a large part of the bigrams appeared in very few posts. This led to the creation of very sparse feature tables, where the information is spread across many columns with zero values. On such data, the models fail to learn stable patterns and performance drops noticeably compared to unigrams, which produce less sparse tables. Larger or more homogeneous datasets could exploit bigrams better, but in this particular project unigrams proved more efficient.

7.4 d. Most frequent words and bigrams by positive and negative sentiment

In the last sub-question of the further analysis, we were asked to identify the most frequent words and bigrams that appear in positive and negative posts. For this purpose, we used the final dataframe that contained the `clean_text` field and the `final_sentiment` label for each post.

The procedure we followed was:

- Filtering the dataset to keep only posts with positive or negative sentiment, ignoring the neutral ones.
- Computing word and bigram frequencies separately for each sentiment category.
- Ranking in descending frequency, so as to identify which words and which bigrams characterise positive and negative sentiment the most.

The results showed that positive sentiment is dominated by words and bigrams related to support, improvement and personal development, such as *thank advance* and *greatly appreciated*, while in negative sentiment the most frequent words and bigrams are related to problems, difficulties and negative experiences, such as *panic attack* and *hard time*. Of course there are also some common words in both categories, for example *like*.

For a better visualisation of the results, we created two bar charts presenting the ten most frequent words and the ten most frequent bigrams in positive and negative sentiment respectively, which clearly show the differentiation in vocabulary between the two categories.

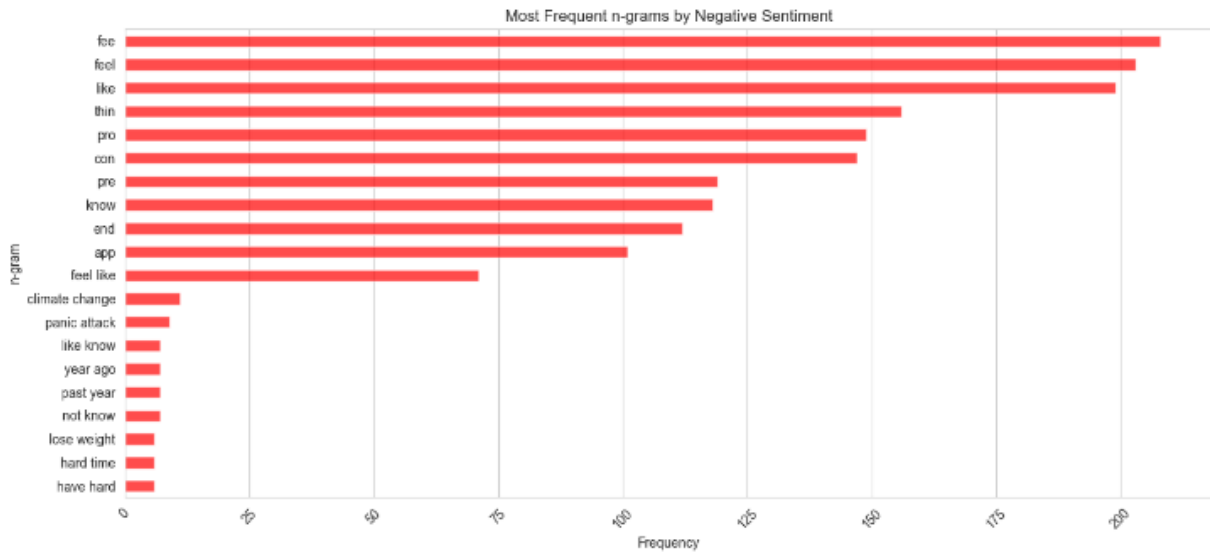


Figure 29: Bar chart with the most frequent words and bigrams for negative sentiment.

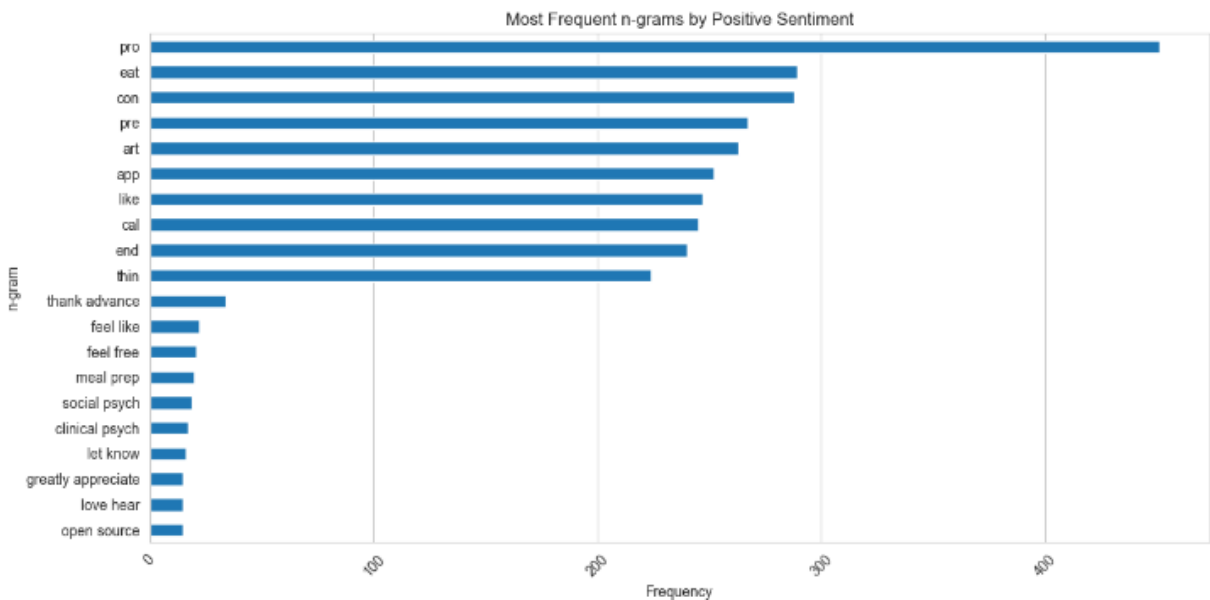


Figure 30: Bar chart with the most frequent words and bigrams for positive sentiment.

Bibliography

- [1] Reddit, “Reddit: Dive into anything.” [Online]. Available: <https://www.reddit.com/>
- [2] P. B. Boe, “PRAW: The Python Reddit API Wrapper,” GitHub repository, 2025. [Online]. Available: <https://praw.readthedocs.io/>
- [3] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, “spaCy: Industrial-strength Natural Language Processing in Python,” *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, Vancouver, Canada, 2017, pp. 85–90.
- [4] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. (draft). Stan-

- ford, CA, USA: Prentice Hall, 2023. [Online]. Available: <https://web.stanford.edu/~jurafsky/slp3/>
- [5] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
 - [6] F. Sebastiani, “Machine learning in automated text categorization,” *ACM Computing Surveys*, vol. 34, no. 1, pp. 1–47, Mar. 2002, doi: 10.1145/505282.505283.
 - [7] S. M. Malakouti, M. B. Menhaj, and A. A. Suratgar, “The usage of 10-fold cross-validation and grid search to enhance ML methods performance in solar farm power generation prediction,” *Cleaner Engineering and Technology*, vol. 15, Art. no. 100664, Aug. 2023, doi: 10.1016/j.clet.2023.100664.
 - [8] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
 - [9] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
 - [10] B. Pang and L. Lee, “Opinion mining and sentiment analysis,” *Foundations and Trends in Information Retrieval*, vol. 2, no. 1–2, pp. 1–135, 2008.
 - [11] A. Agresti, *An Introduction to Categorical Data Analysis*, 3rd ed. Hoboken, NJ, USA: Wiley, 2019.
 - [12] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.

Appendix

The notebooks with the code for this assignment are provided alongside this report.